UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Deep Learning for NLP

Student name: *Vasilakis Nikolaos*
sdi: <->

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Spring Semester 2025*

Contents

1 Abstract **2**

2 Data processing and analysis **2**
2.1 Pre-processing . . . 2
2.2 Analysis . . . 3
2.3 Data partitioning for train, test and validation . . . 7
2.4 Vectorization . . . 8

3 Algorithms and Experiments **9**
3.1 Experiments . . . 9
3.1.1 Table of trials . . . 9
3.2 Hyper-parameter tuning . . . 10
3.3 Optimization techniques . . . 10
3.4 Evaluation . . . 10
3.4.1 ROC curve . . . 11
3.4.2 Learning Curve . . . 12
3.4.3 Confusion matrix . . . 12

4 Results and Overall Analysis **13**
4.1 Results Analysis . . . 13
4.1.1 Best trial . . . 14

5 Bibliography **14**

1. Abstract

This is a report following the attempt to build a sentiment classifier on a given English Twitter data set. The classifier would assign tweets with a sentiment, with 1 representing a positive sentiment and 0 representing a negative sentiment. The sentiment classifier would be developed only using Logistic Regression and only using TF-IDF.

Moreover, each tweet in our database contained an ID, Text and a Label. The ID was used to uniquely identify the tweet, the Text was the Tweet's content, and the Label was indicating whether a tweet had either a positive or a negative sentiment.

2. Data processing and analysis

2.1. Pre-processing

Realizing that the tweets we had contained a lot of information, the idea of preprocessing the data with the aim of cleaning them was necessary. There are 6 techniques that are used to clean the data properly.

- 1. Converting all text to lowercase: This is done because the meaning of the word will be the same regardless of how many capital letters there.
- 2. Removing URLs and mentions (@s): URLs and mentions essentially give no information about the sentiment of a sentence.
- 3. Removing punctuation and non-ASCII characters: Punctuation is removed, apart from exclamation marks (!) and question marks (?) because these two can give information about the sentiment of a sentence. Fullstops, commas, etc. do not give any information about the sentiment of a sentence; therefore, they are removed. Moreover, non-ASCII characters are also removed because they do not contain relative information about a sentence and its sentiment.
- 4. Removing hash tags: Hashtags can indicate if a tweet has a positive or a negative sentiment, however, they are removed for a number of reasons. Firstly, hash tags encode words in an unstructured way (for example, they contain compound words), therefore, our model is likely to not understand the true meaning of the hash tag. Also, hash tags might cause overfitting since they the model treats hash tags as unique features and words, therefore making it much harder to generalize.
- 5. Correction of spelling mistakes: Common spelling mistakes as well as slang is corrected. For instance, the word "gud" is converted to "good", the word "wanna" is converted to "want to", etc. This is done in order to clean up some of the noise that is caused by the data.
- 6. Converting many characters into one: The final technique used, is converting more than 3 characters into a single one. For example, the word "heyyyyy" in a tweet is converted to the word "hey". This is also done in order to clean up noisy data and have a better model.

2.2. Analysis

Along with the data preprocessing, an Exploratory Data Analysis (EDA) is also being done, in order to understand and vizualise the data in a better and more meaningful way. The given EDA graphs are generally on the preprocessed data, however there are graphs from unprocessed data later on, that justify the preprocessing that was done and was mentioned earlier in the report.

The first data analysis that must be done, is whether the data is balanced or not. A balanced data set makes the training of the model easier, because the model will not have any bias towards a particular label. Consequently, the positive and negative instances must be roughly the same.

After the analysis, it is clear that the data set is balanced with 74916 positive instances and 74192 negative instances.

Figure 1: Class distribution in the training data set

Data analysis is also used for the most common words in the already preprocessed data. The most common words are separated between positive and negative instances. The reason this analysis is done is because it allows for better stopword removal, as well as better filtering of overly common words that exist in both positive and negative instances. In short, it is very useful in removing words that exist in both instances and do not help us separate positive and negative instances.

The same analysis is done for bigrams as well. The reason, this analysis is done (combined with data analysis on single unigrams) is because it enhances the understanding of word patterns that appear in positive and negative instances alike. A word might be frequent in both instances, however it may still help determine if a tweet has positive or negative sentiment.

For example, "don't" is a uingram that appears frequently in both positive and negative instances. However bigrams with the word "don't" are much more frequent in negative instances, thus we cannot include the word "don't" in the stop words that will be removed later on.

Below is the visualization of the most frequent words in positive and negative instances seperately.

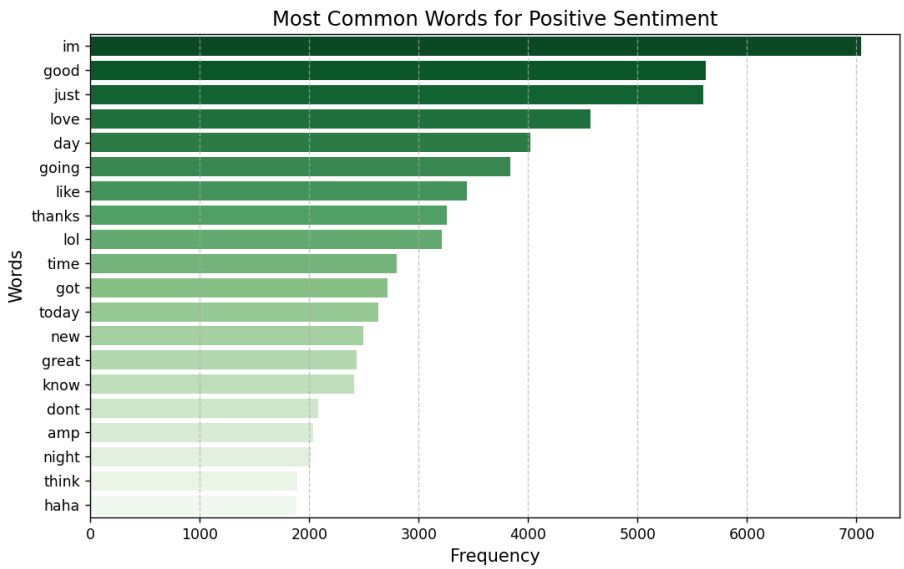

Figure 2: Most Common Words for Positive Sentiment

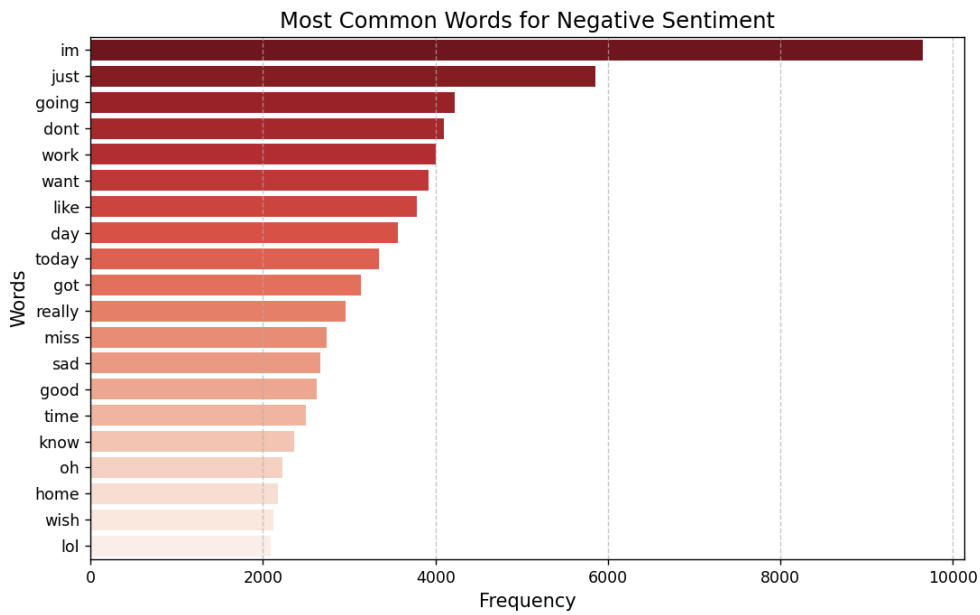

Figure 3: Most Common Words for Negative Sentiment

It is clear that there are words ("i'm", "just", "going", "don't", "day", "got", "like", "today") which appear frequently in positive and negative instances as well.

This is benefitial in order to understand which words might need to be removed later on, since they offer no information on whether a sentence/tweet has a positive or a negative sentiment.

Moving on, the same form of graph is used to analyze bigrams as well. Positive and negative instances are covered seperately

Below is the visualization of the most frequent bigrams in positive and negative instances seperately.

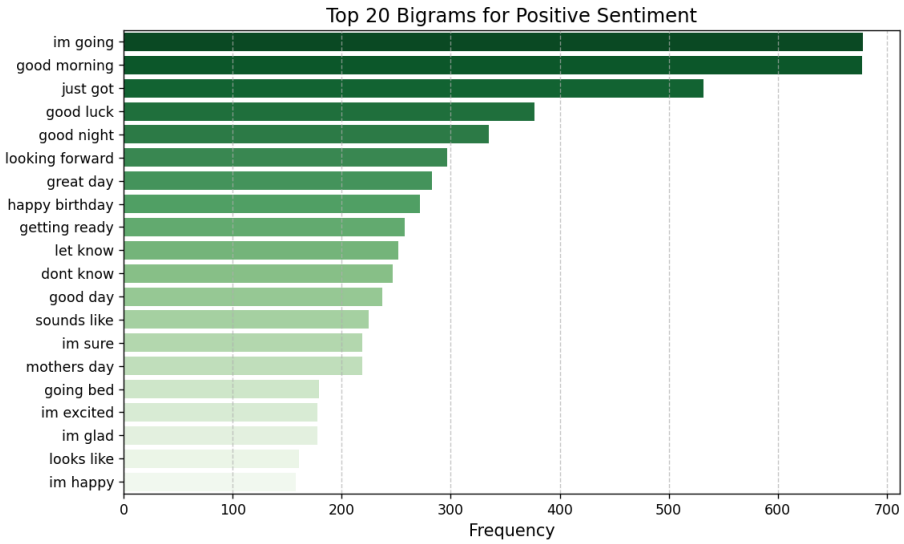

Figure 4: Most Common Bigrams for Positive Sentiment

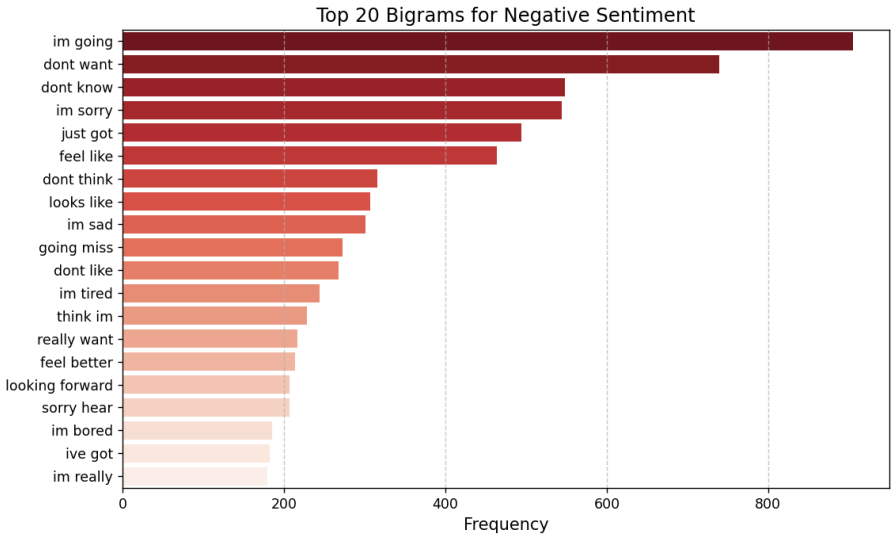

Figure 5: Most Common Bigrams for Negative Sentiment

As we can see bigrams are much more useful for identifying if a tweet has a positive or negative sentiment. Only five expressions are in both graphs ("im going", "just got", "dont know", "looks like", "looking forward") and most of them aren't in the same position in the positive and negative instances. This gives a better instance on which stop words we need to remove later on, when applying the TF-IDF method.

Important note: Data analysis for the most common words was used before preprocessing the data as well The logic behind it was the same (finding interesting patters that help remove words). Even though the most common words didn't have much difference with the ones that appear in Figures 2 and 3, it is interesting to note that the results found on the most common *bigrams* for positive and negative instances before preprocessing the data were quite different

Below is the visualization of the most frequent bigrams in positive and negative instances seperately before the preprocessing

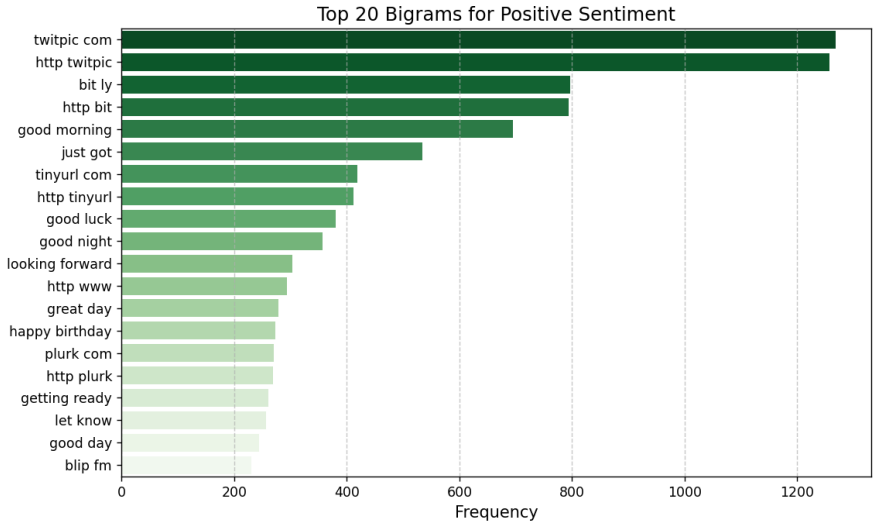

Figure 6: Most Common Bigrams for Positive Sentiment Before Preprocessing

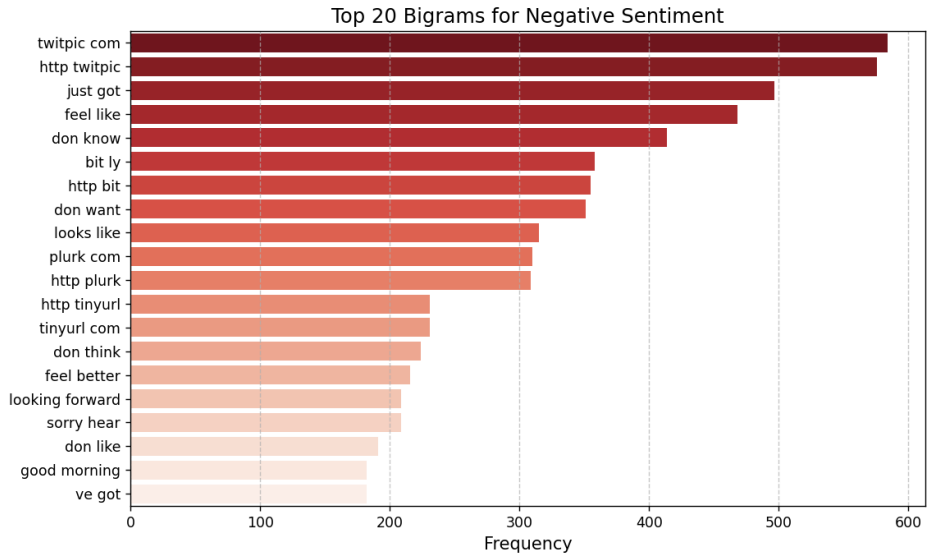

Figure 7: Most Common Bigrams for Negative Sentiment Before Preprocessing

As it is clear, the most common bigrams are just links to other websites, accounts, etc. Thus, the bigrams aren't helpful in determining if a tweet is positive or negative. Since they offer no information and make our model much more complicated, the

URLs and mentions are removed as part of the preprocessing that was mentioned in 2.1

Finally, the last analysis we make is on the length of each tweet and we compare the length of positive and negative tweets in the data set.

Below is is the graph of sentence lengths for positive and negative tweets.

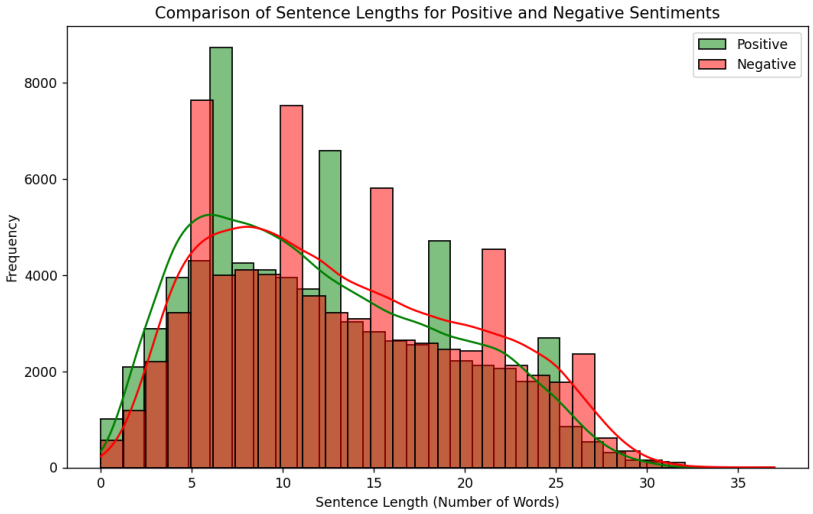

Figure 8: Comparison of Sentence Lengths for Positive and Negative Instances

It is clear that positive instances are a bit shorter than negative ones. The distributions do have a bit of overlap, however tweets with negative sentiments are slightly longer and have more words than tweets with positive sentiments. Unfortunately, this cannot be a feature in our model since we are allowed to only use Logistic Regression and only use TF-IDF. But, it is still important to mention it, since it shows a small but important distinction between positive and negative sentences that cannot be used as a feature in the method that is used.

Note: It is important to mention that the exploratory data analysis was done only on the training data. This is because we aim to avoid any potential influence from the test data, as the test set is meant for evaluating the final model's performance. Performing EDA on the test data could inadvertently lead to biases that would affect the model's generalization ability. For the same reason, no preprocessing was performed on the validation data, as it is used for hyperparameter tuning. Analyzing the validation set could have biased the selection of hyperparameters and skewed the results, leading to overfitting.

2.3. Data partitioning for train, test and validation

The data set was already partitioned when it was given to us. We used a training data set to train our model and tuned it based on the results given from the validation data set. No tuning or optimization was performed on the basis of the test data set. The test data set was only used for the Kaggle competition and in order to see how good is the generalization of the model.

2.4. Vectorization

For vectorization we utilized the Term Frequency-Inverse Document Frequency (also known as TF-IDF) technique in order to convert the tweets into numbers, which can later be fitter into the model of Logistic Regression. TF-IDF is a statistical measure used to evaluate word importance. The formula of TF-IDF is given as follows:

$$TF - IDF(t, d) = TF(t, d) * IDF(t)$$

where:

- TF(t,d) is the number of times a term appears in a document
- IDF(t) is a metric which measures how important a word is across multiple documents. It's formula is:

  $$IDF(t) = log(N/DF(t))$$

  where
  - N is the total number of documents
  - DF(t) is the number of documents containing term t

The benefit of the TF-IDF method is that it reduces the weights of common words such as "the", "is", etc. while also emphasizing unique terms in a document. This is useful for sentiment classification as it ensures that words strongly associated with either positive or negative sentiment are highlighted in an efficient way. Also, it downplays extremely common words which appear often in most tweets and don't give information on whether a tweet has a positive or a negative sentiment.

In our model, TfidfVectorizer was used from the scikit-learn library in Python. The parameters in the TfidfVectorizer were the following:

- ngram_range(1,3): Not only words, but also bigrams and trigrams were used. This was done because bigrams and trigrams capture the essence of a sentence/tweet in a better way and give more understanding of the context of a tweet. 4-grams and above weren't used to avoid overfitting the model because they capture very specific word sequences which may not generalize well.
- max_features(40000): The number of features used needed to be a number which would allow a complex model that had a good accuracy. However, the model couldn't have too many features since this would lead to a large gap between the training and validation accuracy, and thus cause overfitting. 40000 was good number which balanced both of these needs.
- stop_words = stopwords_to_remove: Commonly used words such as "and", "or", "of" as well as some words which were seen in both positive and negative sentiments in the data analysis ("im", "going", "just") are removed since they do not help classify positive or negative sentiments.
- sublinear_tf = True: It applies sublinear term frequency scaling. This is done for the purpose of reducing the impact of frequent term and preventing them from dominating the model.
- min_df = 4: With this parameter, very rare words that appear in less than 4 documents are strained. Very rare words often add noise and might cause our model to be less effective.

3. Algorithms and Experiments

3.1. Experiments

In order to evaluate the accuracy of the classifier the training data set was used to train our model, and then the validation data set to fine tune and optimize our model. The test data set was only used to calculate the final score in the Kaggle competition and see how well the model generalized.

In order to build an efficient classifier, the initial approach that was used was to build the simplest classifier possible. This means that the pre-processing function that was used contained only a few common spelling mistakes as well as converting everything on lowercase. Also, no parameters were used in the functions for the 'TfidfVectorizer' or in the Logistic Regression function. The code that was used was the one from the tutorials by Yorgos Pantis. The initial accuracy of the classifier was approximately 0.79% and the overfit was approximately 5%.

Afterwards, the main focus was the pre-processing function in order to clean the data. A lot of errors and noise appeared in tweets. The steps that were done, are described above in the section 2.1. However, some attempts lead to the accuracy being dropped, such as converting emojis into words, converting numbers into words, and removing question marks and exclamation marks. The final pre-processing function minimally increased the accuracy; however, overfitting was lowered by 1%.

Subsequently, the parameters for the 'TfidfVectorizer' were tuned. The features were intially low - around 10000.

The first optimization that was tried was adding N-grams. The n-gram range that was added was from 1 through 3. The main purpose was to include phrases which would give more information about the sentiment of a sentence. This resulted in an increase of the accuracy by almost 0.007% going from 0.79114 to 0.79859, while overfitting also saw a significant reduction to around 2.5% between the training data set and the validation data set. Attempts at adding 4-grams as well had a minimal effect on the accuracy but increased overfitting. Consequently, 4-grams weren't added in the final model

The second optimization was the removal of stopwords which were very common. This lead to an increase in the accuracy as well. Further improvements were applying sublinear term frequency scaling which improved the accuracy. Furthermore, experiments were made on the parameter 'min_df' which removes words which appear in few documents. The value which was best on the validation data set was min_df = 4. However, all values of the parameter (above 1) increased the accuracy.

Finally, the max features of the vectorizer were increased to 40000 in order to improve accuracy. Also, regularization was added in order improve the generalization of the model. At first Grid Search was considered as the best option in order to find the best hyper parameters, however this was eventually rejected and manual tuning was chosen instead. This is due to manual tuning giving direct feedback through the learning curve about how our model generalizes and allows to balance the flexibility of the model and the cost of overfitting. After numerous attempts, l2 regularization was chosen with the hyperparameter of C = 1.1

3.1.1. Table of trials.

Trial	Score
Minimal preprocessing, no parameters	0.79026
Improved preprocessing	0.79114
Removing question marks and exclamation marks	0.79078
Adding max features = 10k and n-grams	0.79859
Adding min_df and sublinear frequency scaling	0.80022
Improving max features to 40k	0.80366
Hyperparameter tuning	0.80373

Table 1: Summary of trials done to improve accuracy on the model

3.2. Hyper-parameter tuning

To better tune the model, regularization was used. At first the option for Grid Search seemed like the obvious one, in order to find the best hyper parameters for the model. However, it was eventually not chosen as it did not give direct feedback and insight on how well the model generalized because the learning curves weren't seen during each attempt. So, manual tuning was instead preferred which led to a better understanding of the trade-off between the hyper parameter C and the generalization of the model. Eventually, the hyper parameter C = 1.1 was chosen for regularization as it provided the best balance between performance and generalization. Higher values of C were causing overfitting above 5% while lower values of C were causing a high drop in accuracy.

3.3. Optimization techniques

The optimization techniques which were used were the following

- Cross validation: 5-fold validation was used.This was done with the intent of ensuring better generalization as well as model robustness.
- Lasso Regularization: L2 regularization was chosen because it penalizes weights errors and thus helps reduce overfitting.
- Feature selection: Max features = 40000 and min df = 4 were used which helps the model reject overly rare words and also limits it to the most informative ones, therefore it is able to generalize better.

3.4. Evaluation

To evaluate the performance of the model, the evaluation metric that is used is accuracy. Since our dataset is balanced, and there is no priority of false negatives over false positives (or vice versa) as both types of misclassification have similar consequences, accuracy is the preferred metric.

However, the scores of recall, precision and F1-score are given below. F1 score is given by the type:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

and is essentially a metric which balances precision and recall into a single metric. This is done because of the precision/recall trade-off which means that when recall is increased, precision is decreased and vice versa.

Below are the scores of the final model:

```
Accuracy: 0.80373
Precision: 0.79915
Recall: 0.81141
F1 Score: 0.80523
              precision    recall  f1-score   support

           0       0.81      0.80      0.80     21197
           1       0.80      0.81      0.81     21199

    accuracy                           0.80     42396
   macro avg       0.80      0.80      0.80     42396
weighted avg       0.80      0.80      0.80     42396
```

Figure 9: Different evaluation metrics for the final model

The above results show the precision-recall trade-off, which was mentioned earlier. The F1-score is a balance between these two metrics. Since the dataset is balanced, accuracy is also close to the F1-score, reflecting the overall performance without being skewed by class imbalance.

3.4.1. ROC curve. Another useful way to evalute our model is through the ROC curve. The receiver operating characteristic (ROC) curve is a tool used for evaluating the performance of a binary classifier. The ROC curve plots the true positive rate against the false positive rate at different thresholds

The ROC curve of the final model is given below

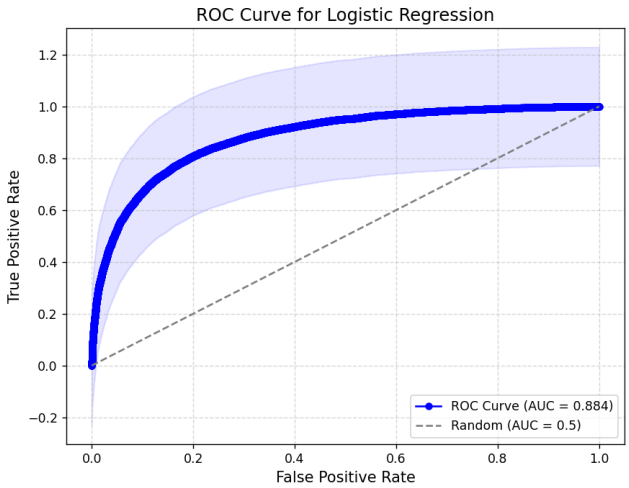

Figure 10: ROC curve for the final model

The ROC curve has an Area Under Curve (AUC) of 0.884 which shows a good ability to discriminate between positive and negative sentiments, compared to a random classifier.

3.4.2. Learning Curve. The learning curves are another good way to evaluate the final model. A learning curve shows the performance of the model against training size. Two curves are shown simultaneously. The curve that concerns the training performance and the curve that concerns the validation/test performance.

Below are the learning curves of the final model.

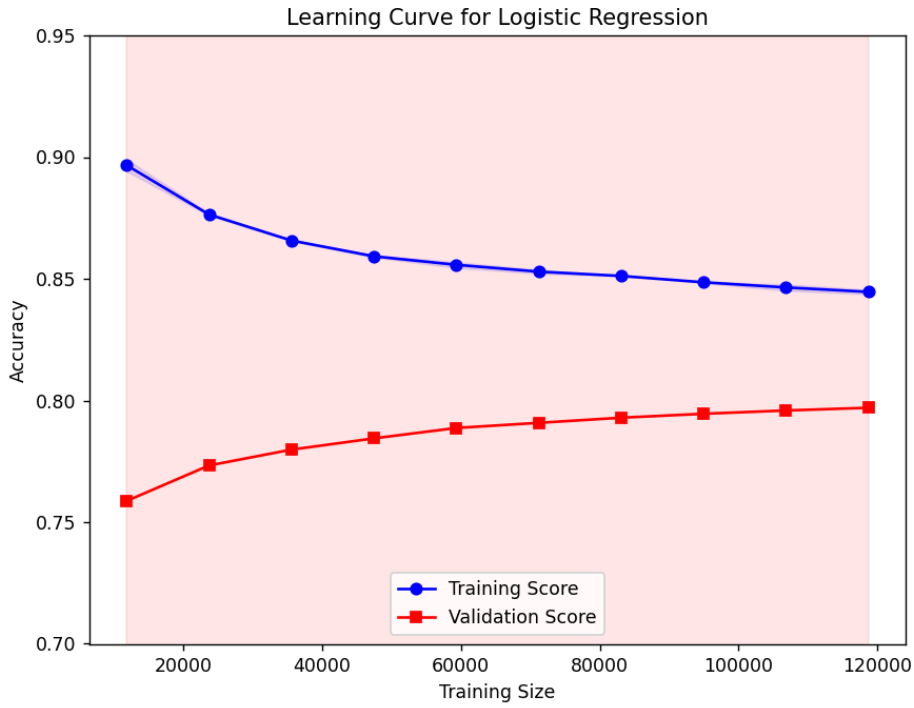

Figure 11: Learning curves for the final model

It is clear that there is overfitting of about 4.5% between the training and the validation score. However the validation curve has an upward trend while the training curve has a downward trend. Moreover, both curves tend to stabilize towards the end of training. Taking all of these into account, as well as the fact that Logistic Regression with TF-IDF is a relatively simple model, the 4.5% value of overfitting was considered acceptable and a good balance between the accuracy provided by the final model and its ability to generalize to unseen data.

3.4.3. Confusion matrix. Finally, a better way to evaluate the performance of our model is the confusion matrix. The confusion matrix works like this: Each row of a confusion matrix represents an actual class, while each column represents a predicted class. Therefore, true positives, true negatives, false positives and false negatives all have a numerical value in the confusion matrix. Therefore, a confusion matrix gives a vizual representation of metrics beyond accuracy such as recall and precision and is useful in identifying possible problems with our model.

Below is the confusion matrix of the final model.

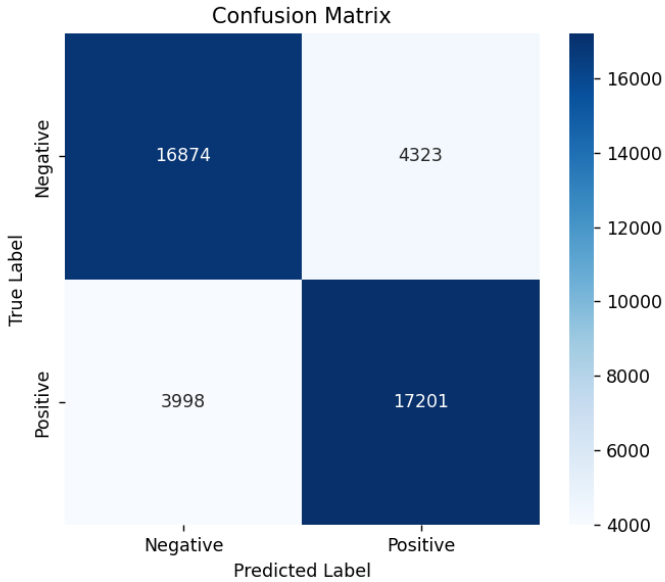

Figure 12: Confusion matrix for the final model

The confusion matrix shows that the number of False Positives is slightly higher than the number of False Negatives. As a result, the recall score is slightly higher than the precision score. However, since the True Positives and True Negatives are significantly higher overall, the model demonstrates high accuracy.

4. Results and Overall Analysis

4.1. Results Analysis

The final model scores an accuracy of 80.373%, while the overfitting gap between the training and the validation data set is around 4.5%. This is considered a fair result given the fact the the model complexity for Logistic Regression with TF-IDF is quite simple (a simple binary classifier).

The confusion matrix and the recall and precision scores indicate that the misclassification of positive and negative instances is pretty balanced. The classifier has a strong ability to indicate whether a tweet has a positive or a negative sentiment. Of course, there are cases in which the classifier does not identify the sentiment correctly, but this is expected due to the nature of the algorithms that are used, as well as the fact that text can sometimes be spelled wrongly, use slang, very rare words, etc.

The integration of 2- and 3-grams into the vectorizer was essential for improved performance of the classifier. This indicates that it is important to integrate phrases and not only words, as they can inform in a better way about the sentiment of a sentence. Concurrently, 4-grams and above are not used, as they can be a cause for overfitting the model more than necessary.

Ideally, all wrong spellings and slang would be replaced with the correct words, in order for less noise to occur in the training data. However it is essentially impossible to include in the pre processing function every possible wrong spelling or every possible slang that would be used in a tweet. Therefore, only common misspellings and

common slang was replaced.

The learning curve and ROC curve show that the classifier is much better at separating positive from negative sentiments on tweets compared to a random one. At the same time, the overfitting of 4.5% is a good balance between the accuracy of the model and its ability to generalize well.

4.1.1. Best trial. The model's best trial achieved accuracy of 80.373% with overfitting of about 4.5%. It is clear that the inclusion of unigrams, bigrams and trigrams help increase the accuracy. Other features include having a sublinear term frequency scaling in order to reduce the impact of frequent terms and also removing a few common words such as "and", "going", "of", etc. Also removing very rare words which appeared in less than 4 documents and having the number of max features to around 40000 helped balance accuracy and reduce overfitting.

Moreover, a big part in making the model effective was the pre-processing function which included converting every letter to lowercase, correcting common spelling mistakes and converting 'slang' terms to their actual meaning. Additionally, removing URLs, mentions, hashtags, non-ASCII characters and punctuation (except for exclamation marks and question marks) increased accuracy. Converting more than 3 characters into one (e.g. converting 'heyyyy' into 'hey') was beneficial as it reduced noise and cleaned the data.

Finally, Lasso regularization was applied with a hyperparameter of C = 1.1 as it provided the best balance between generalization and performance.

5. Bibliography

References

[1] Classification: Roc and auc, 2024. https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc.

[2] Manolis Koubarakis. Artificial intelligence ii course homepage for the department of informatics and telecommunications, 2025. https://cgi.di.uoa.gr/~ys19/.

[3] Manolis Koubarakis. Backpropagation, 2025. https://cgi.di.uoa.gr/~ys19/lectures25/backpropagation.pdf.

[4] Manolis Koubarakis. Introductory concepts of machine learning, 2025. https://cgi.di.uoa.gr/~ys19/lectures25/introductory_concepts.pdf.

[5] Manolis Koubarakis. Regression, 2025. https://cgi.di.uoa.gr/~ys19/lectures25/regression.pdf.

[6] Yorgos Pantis. Tutorial for homework 1, 2025. https://cgi.di.uoa.gr/~ys19/tutorials25/HW1_lab_AI2_25.pdf.

You can refer to the material from [1], [2], [3], [4], [5] and [6].